Encrypt-then-MAC vs. ChaCha20-Poly1305 AEAD: Implementation and Performance Analysis

Juan Camarero de la Torre

UID: 121234537

A. James Clark School of Engineering, University of Maryland

Electrical Engineer

Gorjan Alagic

May 1, 2025

Introduction and Background

Modern secure communication necessitates authenticated encryption (AE), which guarantees both confidentiality (only authorized parties can access the message) and integrity (any unauthorized tampering is discernible). In classical approaches, a symmetric encryption algorithm ensures confidentiality, while a Message Authentication Code (MAC) ensures integrity. However, combining these approaches incorrectly can compromise security; for instance, the outdated TLS approach of MAC followed by encryption suffered from vulnerabilities such as padding oracle attacks. Two robust strategies for AE are: (1) employing a generic composition of encryption and MAC (specifically, the encrypt-then-MAC (EtM) approach, which is widely recommended), and (2) utilizing an integrated Authenticated Encryption with Associated Data (AEAD) scheme, which integrates encryption and authentication in a single step, obsoleting the distinction between "MAC-then-encrypt" and "encrypt-then-MAC."

This project delves into the exploration and implementation of two distinct encryption schemes: AES-CBC encryption coupled with HMAC-SHA256 authentication (Encrypt-then-MAC) and ChaCha20-Poly1305 (an AEAD scheme). Each scheme's design decisions and security properties are comprehensively documented within our implementations. Furthermore, we conduct simulations of tampering attacks to comprehensively assess system behavior and evaluate the performance of these schemes. Illustrative code snippets, utilizing Python's cryptography library, are provided to elucidate the implementations and tests conducted. Conclusively, we present the findings and draw a comparative analysis of the advantages and disadvantages of both approaches.

Encrypt-then-MAC

Encrypt-then-MAC (EtM) entails encrypting the plaintext with a symmetric cipher and subsequently computing a MAC on the encrypted data. The ciphertext and the MAC tag are transmitted together. The recipient first verifies the MAC; only if the tag is valid will they proceed with decrypting the ciphertext. This sequential process ensures that any modifications to the ciphertext (including the initialization vector (IV) or any associated data) will be detected before decryption, effectively thwarting chosen-ciphertext attacks. Our implementation employs AES in CBC mode for encryption and HMAC-SHA256 for the MAC. AES (Advanced Encryption Standard) is a 128-bit block cipher widely adopted as a secure encryption standard (FIPS-197). CBC (Cipher Block Chaining) mode XORs each plaintext block with the preceding ciphertext block prior to encryption; it necessitates a unique random IV for each message to guarantee distinct ciphertexts. HMAC-SHA256 is a hash-based MAC utilizing SHA-256, providing a 256-bit authentication tag. We utilize a 256-bit key for AES (AES-256) and a separate 256-bit key for HMAC. Separating keys for encryption and MAC is a design choice to prevent potential key-reuse issues and aligns with standard practice for EtM.

Design decisions

- Key management: We employ independent keys for AES and HMAC. These keys can be derived from a master key or password; however, for demonstration purposes, our code generates random keys.
- IV handling: A fresh 128-bit IV is randomly generated for each encryption. The IV is included in the MAC computation (i.e., we MAC over IV || ciphertext) and transmitted along with the ciphertext and tag. Including the IV in the MAC prevents an attacker from altering the IV undetected, which is crucial because in CBC mode, flipping bits in the IV would deterministically flip bits in the first plaintext block after decryption.
- Padding: AES-CBC operates on 16-byte blocks, necessitating the use of PKCS#7 padding to accommodate plaintexts that do not align with the block size. The receiver will remove the padding upon decryption.
- Encryption and MAC algorithms: AES-256 offers robust security capabilities (128-bit block, 256-bit key), while HMAC-SHA256 provides a secure MAC with a 256-bit output. This combination is designed to deliver a high security level, suitable for safeguarding data up to 256-bit security strength, provided that the underlying components are secure.
- Operation order: We adhere strictly to the established procedure: compute the ciphertext, followed by the MAC. During decryption, we verify the MAC first, and then proceed with the decryption. This order ensures that the decryptor remains protected from maliciously altered ciphertexts.

AEAD ChaCha20-Poly1305

The second scheme exemplifies an Authenticated Encryption with Associated Data (AEAD) algorithm, specifically ChaCha20-Poly1305. AEAD algorithms seamlessly integrate encryption and authentication into a unified process. ChaCha20-Poly1305, introduced as an Internet standard (originally in RFC 7539, subsequently updated by RFC 8439), has been adopted into protocols such as TLS 1.3, SSH, and WireGuard. It comprises the ChaCha20 stream cipher for encryption and the Poly1305 Message Authentication Code (MAC) for authentication. Additionally, it accommodates optional associated data (AAD), which is authenticated but not encrypted (e.g., protocol headers).

ChaCha20 is a 256-bit key cipher that generates a pseudorandom keystream, processing data in 64-byte blocks. It operates based on simple add-rotate-xor (ARX) operations and does not necessitate specialized hardware for efficiency. Poly1305 is a high-speed authenticator that produces a 128-bit tag. It is designed to be one-time, requiring a unique Poly1305 key for each message, which, in this construction, is derived from the ChaCha20 keystream.

The ChaCha20-Poly1305 construction defined in RFC 8439 utilizes a single 256-bit key, split into a 256-bit ChaCha key and a 128-bit Poly1305 key. The Poly1305 key is obtained by encrypting a 128-bit zero block with ChaCha20 using the message nonce and main key. This construction requires a 96-bit nonce per message and produces a 16-byte authentication tag. No padding is necessary for the message, as ChaCha20 operates on arbitrary-length input by XORing a keystream with the plaintext, similar to how AES operates in counter mode.

Design decisions

- Key management: We employ a 256-bit key for ChaCha20-Poly1305. This contrasts with the previous scheme, which utilized two distinct keys. In this instance, a single key is responsible for both encryption and authentication. (Internally, the algorithm generates sub-keys.)
- Nonce: To guarantee the uniqueness of each encryption operation with the same key, a 96-bit (12-byte) nonce is mandated by the algorithm. We opt to generate a random nonce for each message. The probability of a collision is exceedingly low when nonces are truly random (the birthday bound for 96-bit is approximately 2^{48} messages). However, for absolute security in a real system, a counter may be employed to guarantee uniqueness. Reusing a nonce with ChaCha20-Poly1305 is catastrophic, potentially compromising confidentiality and integrity entirely (comparable to the revelation of information through the reuse of an IV in AES-GCM/CTR). Consequently, a design decision is to treat the nonce as a counter or a random value that never repeats for a given key. In our demonstration code, we include the nonce alongside the ciphertext and tag for the recipient.

- Associated data: Our implementation supports the handling of associated data, such as headers that require integrity protection but not encryption. However, we emphasize that including AAD is achieved by passing an additional bytes object that will be protected by Poly1305 but not encrypted by ChaCha20.
- Library usage: We leverage a sophisticated Advanced Encryption Standard (AES) Authenticated Encryption with a tag (AEAD) API from the cryptography library. This API efficiently manages the generation and verification of Poly1305 tags, thereby mitigating the potential for misuse.

Security comparison

Both the AES-CBC-HMAC (EtM) scheme and the ChaCha20-Poly1305 AEAD scheme provide authenticated encryption, yet they differ in their security considerations and applications.

- Theoretical Security: When implemented correctly, both schemes provide IND-CCA2 security (indistinguishability under chosen-ciphertext attacks), ensuring that an attacker cannot decrypt or forge messages. Encrypt-then-MAC, employing a secure cipher and MAC, is provably an AE (Authenticated Encryption) scheme. ChaCha20-Poly1305 is also proven secure (under certain models) given the security of ChaCha20 and Poly1305, as well as the uniqueness of nonces. In essence, both meet the definition of authenticated encryption. However, the security of either scheme can be compromised if its prerequisites are violated, such as reusing the initialization vector (IV) or nonce, using weak keys, or encountering a flaw in the underlying primitives.
- Reliance on Primitives: The AES-CBC-HMAC approach utilizes two distinct primitives: AES for encryption and SHA-256 (in HMAC mode) for authentication. AES is a well-established standard, while HMAC-SHA256 is also widely regarded as highly secure. ChaCha20-Poly1305, on the other hand, relies on the ChaCha20 stream cipher and Poly1305 MAC. ChaCha20 and Poly1305 were designed by Daniel J. Bernstein between 2005 and 2008 and have undergone extensive cryptanalysis. Both sets of primitives are widely trusted. At the 256-bit security level, neither has been identified with any known practical attacks.

  A notable distinction lies in the fact that AES is a block cipher (with 128-bit blocks), whereas ChaCha20 is a stream cipher (effectively employing a 512-bit block function to generate a keystream). Poly1305, in contrast, is a one-time MAC that necessitates a unique key per message (provided by the cipher in this AEAD construction), whereas HMAC allows for the reuse of its key for multiple messages safely.
- Ease of Correct Implementation: The integrated AEAD (ChaCha20-Poly1305) simplifies the correct implementation for developers, as the algorithm/library handles most of the complexity. In contrast, AES-CBC + HMAC presents more potential for errors: including the IV in the MAC, using proper padding, generating independent keys, and verifying before decryption are crucial steps. Any incorrect execution of these steps can compromise security. For example, forgetting to MAC the IV or performing MAC-then-encrypt instead could introduce vulnerabilities. The AEAD API, on the other hand, automatically handles these details (nonce management still requires careful attention, but it is a single value to manage). Therefore, from an engineering perspective, ChaCha20-Poly1305 (or any modern AEAD like AES-GCM) reduces the margin of error. This is one reason the industry has favored AEAD modes in protocols: they standardize secure practices.

- Attack Surface: Both encryption schemes provide protection against ciphertext modification and forgery. However, historically, AES-CBC has been susceptible to specific attacks when used without proper authentication, such as padding oracle attacks on TLS where the MAC was verified after decryption. Our EtM design mitigates these attacks by verifying first. ChaCha20-Poly1305, when used with unique nonces, does not have known analogous vulnerabilities. Additionally, ChaCha20 as a stream cipher does not encounter padding issues. Furthermore, ChaCha20-Poly1305 is believed to offer some advantages in resisting timing attacks. AES (in certain modes) can be sensitive to timing if not implemented using constant-time techniques (particularly AES software without hardware support and certain MACs). In contrast, ChaCha20's arithmetic and Poly1305's mathematical operations can be implemented in constant time on general-purpose CPUs. While reputable libraries implement both in constant time, it is worth noting this distinction.
- Associated Data: ChaCha20-Poly1305 (and AEADs in general) provide built-in support for associated data, which is data that requires integrity or authentication but not encryption. In our AES-CBC-HMAC scheme, we could achieve a similar objective by incorporating any associated data into the HMAC computation (prepending or appending it before computing the tag). We did include the IV in the MAC; similarly, additional headers could be MAC'd. However, this approach lacks the same level of standardization and simplicity as AEAD, which directly supports AAD through its API. Therefore, AEAD offers a superior advantage in incorporating associated plaintext, such as protocol headers, in a clear and structured manner.
- Flexibility and Standards: AES-CBC-HMAC offers flexibility in terms of cipher selection and hash functions, but it is not a single standardized algorithm. Instead, it comprises two distinct components. ChaCha20-Poly1305, on the other hand, is a standardized algorithm (RFC 8439) with fixed components. Both approaches are trustworthy, but employing a standard AEAD (either ChaCha20-Poly1305 or AES-GCM) ensures its thorough testing as a unit. Our AES-CBC-HMAC scheme, similar to the construction used in IPsec and other historical systems, is also well-vetted. However, it necessitates aligning multiple standards (AES, HMAC, etc.).

In summary, both security schemes are robust, but the ChaCha20-Poly1305 AEAD offers greater ease of use and practical advantages, such as the elimination of padding requirements, the inclusion of a built-in AAD, and potential resilience to certain side-channel attacks. The AES-CBC-HMAC scheme, when implemented meticulously (as exemplified by the EtM method and adherence to all relevant details), provides comparable security measures. Both schemes safeguard against tampering and eavesdropping under the assumption of the cryptographic strength of their respective components.

Conclusion

In this project, we implemented and analyzed two approaches to authenticated encryption: a conventional encrypt-then-MAC scheme employing AES-CBC and HMAC-SHA256, and a contemporary AEAD scheme, ChaCha20-Poly1305. Both implementations demonstrated their efficacy in safeguarding the confidentiality and integrity of messages, as evidenced by our tampering experiments. These experiments reliably detected and prevented any modifications to ciphertext or tags, thereby ensuring the protection of plaintext.

Pros and Cons Summary:

AES-CBC + HMAC (EtM): This scheme utilizes well-established components. Its advantages include conceptual clarity (separate encryption and authentication steps) and flexibility (the ability to substitute AES with another cipher or HMAC-SHA256 with another MAC as required). It is also widely supported and has a historical presence in various applications, such as IPsec and TLS 1.2 with the EtM extension. When implemented in the recommended encrypt-then-MAC order, it provides robust security. The disadvantages primarily relate to potential implementation challenges and performance overhead: proper padding management, handling of two keys, and ensuring the inclusion of all necessary components (IV, associated data) in the MAC are essential. While there is a slight performance cost associated with two passes (one for encryption and one for MAC), this cost can be minimized with optimized libraries. If not implemented meticulously, this scheme can be vulnerable to attacks. For instance, a mistake such as verifying the MAC after decryption or omitting verification altogether would render it susceptible to serious attacks. In our design, we have mitigated these risks through strict ordering and the inclusion of the IV in the MAC. Another minor disadvantage is message expansion, as the 16-byte IV and 32-byte HMAC tag must be transmitted along with the ciphertext. For ChaCha20-Poly1305, a 12-byte nonce and 16-byte tag are sent, resulting in a slightly reduced overhead.

ChaCha20-Poly1305 AEAD: ChaCha20-Poly1305 AEAD offers several advantages. It is user-friendly, requiring only a single function call to encrypt data, reducing the risk of errors. Additionally, it performs well on a diverse range of hardware, including those lacking AES acceleration. Furthermore, it facilitates the easy handling of associated data without the need for padding. This standardized cipher suite is widely adopted in modern systems, enhancing its security and interoperability.

ChaCha20-Poly1305 also possesses resilience advantages, such as being designed to withstand timing attacks in software implementations. However, its disadvantages are limited. Users must avoid reusing nonces with the same key, which is a general requirement for counter-mode encryption. Additionally, if a system already utilizes hardware-accelerated AES, employing AES-GCM (another AEAD) may yield even

higher encryption speeds. Consequently, ChaCha20-Poly1305's advantages are context-dependent.

Lastly, ChaCha20-Poly1305's components are fixed, which may not be considered a disadvantage. However, it offers less flexibility compared to systems that allow for the use of alternative hash functions, although this is typically unnecessary.

In summary, both encryption schemes successfully achieved the objective of authenticated encryption. The AES-CBC-HMAC EtM scheme demonstrated the significance of meticulous execution in securing data, mirroring the process of constructing an AE scheme from scratch employing cryptographic primitives. Conversely, the ChaCha20-Poly1305 scheme showcased the advantages of contemporary AEAD constructions in terms of user-friendliness and computational efficiency. In the context of real-world deployment, a standardized AEAD such as ChaCha20-Poly1305 (or AES-GCM, contingent upon hardware compatibility) would be preferred due to its simplicity and demonstrated usage in protocols. Nevertheless, the implementation of Encrypt-then-MAC provided valuable insights into the intricacies of authenticated encryption. It underscores the rationale behind employing Encrypt-then-MAC as the recommended approach for custom AE scheme design and the internal structure of integrated designs like ChaCha20-Poly1305, which is characterized by an EtM fashion. Notably, ChaCha20-Poly1305 effectively encrypts the plaintext, followed by MACing the ciphertext and AAD using Poly1305.

References

1. Bertoni, G., Daemen, J., Peeters, M., & Assche, G. V. (2011). *Keccak* (NIST FIPS 202 Secure Hash Algorithm 3 Proposal). (Not directly used in this project, but relevant as cryptographic background.)

2. Gutmann, P. (2014). *Encrypt-then-MAC for Transport Layer Security (TLS) and Datagram TLS* (RFC 7366). Internet Engineering Task Force. doi: 10.17487/RFC7366.

3. Nir, Y., & Langley, A. (2018). *ChaCha20 and Poly1305 for IETF Protocols* (RFC 8439). Internet Research Task Force. doi: 10.17487/RFC8439.

4. Sullivan, N. (2015, February 23). *Do the ChaCha: better mobile performance with cryptography*. Cloudflare Blog. Retrieved from https://blog.cloudflare.com/do-the-chacha-better-mobile-performance-with-cryptography/

5. The Python Cryptographic Authority (2020). *PyCA Cryptography Library (Version 3.4.8)* [Software]. Retrieved from https://cryptography.io/.